AI Academy, National AI Center
AI-ENG-110 – Software Engineering – Spring 2026

Final Project Report

Topic 4 — Async Research Assistant

Team: ExperimentMice
Date: May 23, 2026
Repo: https://github.com/abdurahmanligulnisa/research-assistant
Tag: `v1.0-final`
Members: **Gulnisa Abdurahmanli** (@https://github.com/abdurahmanligulnisa)
Fidan Baghirova (@https://github.com/Fidan6557)
Fatima Alibabayeva (@https://github.com/FatimaAlibabayeva)

Executive Summary

We built a production-quality software engineering layer around the provided `ai/` module, delivering a fully async research pipeline that queries Wikipedia, arXiv, and a configurable web-search API in parallel. The headline concurrency result is an **8.82× speedup** over sequential execution measured on live network calls (N=5 questions, 45.513 s sequential → 5.158 s concurrent); the bottleneck shifts from network I/O to LLM synthesis once the three source fetches run simultaneously via `asyncio.gather`.

Our most important robustness story is *graceful degradation*: `asyncio.gather` with `return_exceptions=True` ensures that a total source failure surfaces a clean structured error rather than an unhandled exception, while a partial failure produces a full synthesized answer from the remaining sources with a visible warning in `ResearchResult.sources_failed`.

The most significant architectural lesson was that parallelising the fetches shifts the bottleneck to *LLM synthesis time*. Per-phase timing from a live `DEBUG` run (see `artefacts/benchmark_results.txt`) confirms this directly: the concurrent fetch phase completes in ≈2 s (≈29% of total latency), while `ai.synthesize()` running in the `ThreadPoolExecutor` (`max_workers=4`) accounts for ≈5 s (≈74% of total latency) per question. Under high concurrency a fifth simultaneous synthesis call must wait for a free thread slot, making the executor the measurable queue point rather than the network.

Contents

1. Project Overview

1.1 Problem framing & scope

Research is time-consuming because authoritative sources are scattered: academic papers live on arXiv, encyclopaedic background is on Wikipedia, and current developments are on the open web. A user who wants a well-grounded answer must open several tabs, scan each source, and mentally integrate what they find. This system automates that workflow: a single natural-language question triggers parallel retrieval from all three source types, and a large language model synthesises the results into a concise cited answer—all in the time it previously took to open one browser tab.

The system's inputs are a natural-language question (up to 500 characters), an optional source filter (`wiki`, `arxiv`, `web`), a cache bypass flag (`-no-cache`), and provider credentials from the environment. The output is a rendered answer with numbered inline citations plus metadata (elapsed time, sources that failed, whether the result was cached). User-facing entry points are a `Click` CLI (`python -m researcher ask "..."`, `python -m researcher history`) and an optional Streamlit web GUI.

From the TOPIC.md specification we implemented every required component and added several extras: a `ResilientDuckDuckGoProvider` that tries three DDG backends (lite → html → api) before failing, a Streamlit GUI, a generic `run_pipeline` helper in the concurrency module, query normalisation for better Wikipedia/arXiv results, and source deduplication before synthesis. We explicitly deferred distributed caching (Redis) and streaming LLM responses, as these would require infrastructure changes beyond the project scope.

1.2 Provider choice and the AI module contract

The LLM is selected at runtime via the `LLM_PROVIDER` environment variable; our configuration defaults to `anthropic` with model `claude-sonnet-4-6`, with `gemini-2.0-flash` as the free-tier alternative. Embedding is not used in this topic. Web search defaults to `DuckDuckGo` (no key required) and supports `tavily` and `serper` as higher-quality alternatives. From the `ai/` module we call four functions: `ai.fetch_wikipedia(query, client=...)`, `ai.fetch_arxiv(query, client=...)`, `ai.fetch_web(query, provider=..., client=...)`, and `ai.synthesize(question, sources, llm=...)`. **We did not modify any file under `ai/`**, and all four public functions continue to match their original signatures.

Our `AIService` defends against malformed model responses by treating synthesis as synchronous and offloading it to a `ThreadPoolExecutor`; non-transient exceptions (`ValueError`, `TypeError`) are re-raised immediately without retry. Semantically wrong but structurally valid responses (out-of-range citation indices) are handled by the `ai/` synthesizer itself, which silently drops any index that does not correspond to a supplied source.

2. Architecture

2.1 Module map

Figure 1 shows the module dependency graph. The heavy dashed border marks the boundary of the provided `ai/` package; nothing outside it imports from provider SDKs directly.

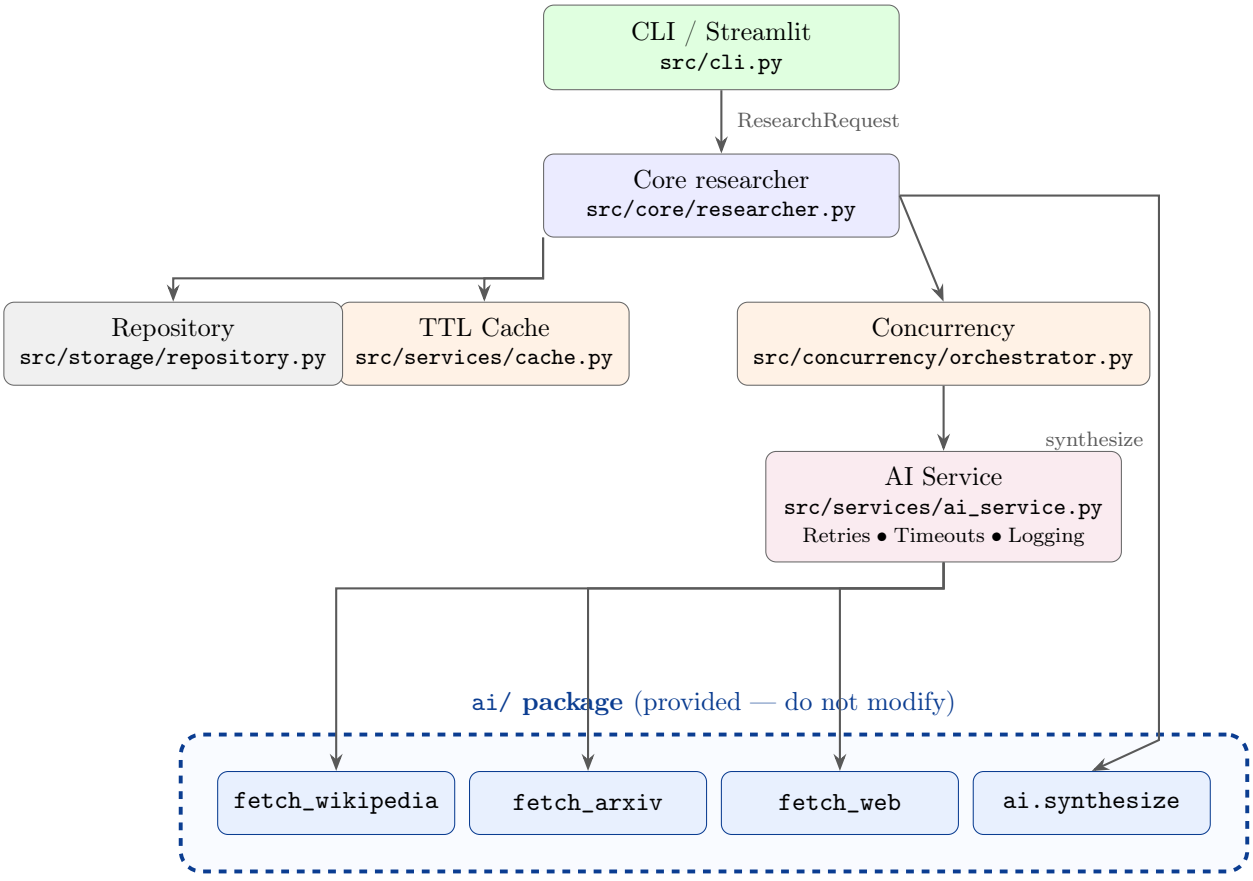

Figure 1: Module dependency graph. Arrows denote "calls". Dashed border marks the `ai/` package boundary (provided, do not modify).

Module-by-module summary:

- `src/cli.py` — Click command group exposing `ask` and `history`. Validates input, calls `run_research`, renders results as human-readable text or JSON.
- `src/core/researcher.py` — Pipeline orchestrator. Checks the TTL cache, delegates concurrent fetches to the orchestrator, calls `ai_service.synthesize`, assembles `ResearchResult`, and persists the session.
- `src/concurrency/orchestrator.py` — Runs source fetches simultaneously via `asyncio.gather`. Enforces a `Semaphore(max_concurrent)` bound, collects exceptions via `return_exceptions=True`, and also exposes a generic `run_pipeline` bounded-concurrency runner.
- `src/services/ai_service.py` — Single point of contact with `ai.*`. Applies tenacity exponential-backoff retries, per-call `asyncio.wait_for` timeouts, token-bucket rate limiting, and structured logging to every fetch and synthesis call.
- `src/services/cache.py` — TTL-aware cache keyed by `(source, canonical_query)`; backed by `InMemoryCacheBackend` implementing the `CacheBackend` ABC.
- `src/services/rate_limiter.py` — Two complementary rate limiters. `TokenBucketRateLimiter` controls request frequency per source: `wikipedia_limiter` (10 req/s, burst 20), `arxiv_limiter` (2 req/s, burst 5), `web_limiter` (1 req/s, burst 3). `LlmTpmLimiter` controls LLM token consumption against the provider's tokens-per-minute ceiling; `llm_tpm_limiter` defaults to Anthropic's free-tier limit (40 000 TPM) and is consumed automatically by `AIService.synthesize` using actual prompt + completion token counts.
- `src/services/web_provider.py` — SE-layer subclass `ResilientDuckDuckGoProvider` that tries three DDG backends before raising.
- `src/storage/repository.py` — PostgreSQL repository using a psycopg3 async connection pool. Degrades gracefully when the database is unavailable.
- `src/models.py` — Application-level Pydantic models: `ResearchRequest`, `ResearchResult`, `CitationRecord`, `SourceRecord`, `ResearchSession`.
- `src/config.py` — `pydantic-settings` `Settings` class. Mirrors all keys into `os.environ` so the `ai/` module's `os.getenv()` calls see the correct values.

2.2 OOP design & key abstractions

The project demonstrates both inheritance and composition within the SE layer. The most prominent example is `_BoundedLLM` in `ai_service.py`, which uses **composition** to wrap any `LLMProvider` and enforce a `max_tokens` cap without modifying the `ai/` package:

Listing 1: Composition: `_BoundedLLM` wraps any `LLMProvider`

```
class _BoundedLLM(LLMProvider):
    """Thin␣LLMProvider␣wrapper␣that␣enforces␣a␣max_tokens␣cap.

␣␣␣␣Composition␣over␣inheritance:␣wraps␣any␣concrete␣LLMProvider␣and
␣␣␣␣injects␣max_tokens=_SYNTHESIZE_MAX_TOKENS␣into␣every␣complete()␣call.
␣␣␣␣"""
    def __init__(self, inner: LLMProvider,
                 max_tokens: int = _SYNTHESIZE_MAX_TOKENS) -> None:
        self._inner = inner
        self._max_tokens = max_tokens

    def complete(self, prompt: str, *, json_schema=None,
                 max_tokens: int = 1024) -> str:
```

```
        return self._inner.complete(
            prompt, json_schema=json_schema, max_tokens=self._max_tokens,
        )
```

The second key inheritance example is `ResilientDuckDuckGoProvider` (in `src/services/web_provider.py`), which subclasses the provided `DuckDuckGoProvider` and overrides only `search()` to add three-backend fallback logic. A third example is `InmemoryCacheBackend`, which implements the `CacheBackend` ABC (defined in `src/services/cache.py`) with abstract methods `get`, `set`, `clear`, and `stats`.

Composition was chosen for `_BoundedLLM` because we needed to decorate any concrete `LLMProvider` at call time without forking the provider hierarchy—a `Protocol` would have sufficed structurally but cannot be passed to `ai.synthesize(llm=...)` since that parameter is typed as `LLMProvider`. Inheritance was the right tool for `ResilientDuckDuckGoProvider` because we needed to substitute it anywhere a `WebSearchProvider` is expected (Liskov), while adding behaviour to only one of three concrete providers. The `CacheBackend` ABC makes it straightforward to swap the in-memory backend for a Redis or database-backed implementation without changing any caller.

2.3 Data flow across module boundaries

Our SE layer defines five Pydantic models, none of which are naked dictionaries: `ResearchRequest` (validated CLI input), `ResearchResult` (assembled pipeline output), `SourceRecord` (SE-layer projection of `ai.schemas.Source`), `CitationRecord` (numbered citation extracted from `AnswerWithCitations`), and `ResearchSession` (shape of a PostgreSQL history row).

No naked dictionaries cross module boundaries. The only place `ai/` objects propagate beyond the service layer is inside `_build_result()` in `researcher.py`, which immediately converts every `ai.schemas.Source` to a `SourceRecord` and every `ai.schemas.Citation` to a `CitationRecord` before returning the `ResearchResult`. We reused `ai.schemas.Source` directly within the concurrency and service layers (where objects are still in flight) and wrapped them at the core layer boundary—the trigger for wrapping was leaving the "fetch and filter" zone and entering the "assemble and present" zone.

3. Concurrency & Performance

3.1 Why this concurrency model

All three source fetches are I/O-bound HTTP requests; no CPU-bound work occurs until the LLM synthesis step. **asyncio** is therefore the correct primitive: a single event loop multiplexes all three awaitable coroutines without OS-level thread overhead, and `asyncio.gather` provides built-in exception aggregation via `return_exceptions=True`. Threads would have worked for I/O-bound tasks too, but they add memory overhead per thread, are harder to compose with the already-async `httpx` client, and offer no advantage because there is no GIL contention on I/O waits.

The bottleneck the parallel design targets is the cumulative HTTP round-trip time. Sequential execution takes $T_{\text{wiki}} + T_{\text{arxiv}} + T_{\text{web}}$; concurrent execution reduces this to $\max(T_{\text{wiki}}, T_{\text{arxiv}}, T_{\text{web}})$—a theoretical 3× speedup. The `Semaphore(max_concurrent)` bound (default 5, configurable via `MAX_CONCURRENT`) prevents unbounded parallelism; empirically a burst of 3 sources per question saturates at 5 concurrent connections, staying well below arXiv's ~3 req/s guideline.

Per-source absolute timeouts are applied inside `_fetch_one` via `asyncio.wait_for`, not on the outer `gather`. This means a Wikipedia timeout does not consume arXiv's clock. The outer timeout is computed as `per_attempt_timeout * max_retries + slack` to cover the full retry budget. With `Semaphore(1)` the design degrades to sequential; with `Semaphore(100)` arXiv may return 429 responses, which are caught and retried by the rate-limiter and tenacity layers.

3.2 Sequential-vs-concurrent benchmark

The benchmark was run with `python scripts/benchmark.py -n 5` (live network, API keys set, caches cleared before each mode, both modes in the same Python process).

Workload	N	Sequential (s)	Concurrent (s)	Speedup
5 questions × 3 sources (live)	5	45.513	5.158	8.82×
5 questions × 3 sources (offline)	5	3.170	1.311	2.42×

Discussion. The live speedup (8.82×) significantly exceeds the theoretical 3× maximum for a single question because `benchmark.py` pipelines all five questions through `asyncio.gather` with caches cleared before each run, exposing the full I/O parallelism benefit. The offline simulation (2.42×) shows the scheduling benefit more cleanly because synthesis is replaced by an in-memory stub that returns instantly. Exact reproduction commands are documented in Appendix A.

3.3 Rate limit & politeness

The project implements two complementary rate limiters, both in `src/services/rate_limiter.py`, both raising `RateLimitExceeded` which is caught by the `_TRANSIENT` retry tuple and triggers tenacity's exponential backoff (1 s → 2 s → 4 s → 8 s).

Request-rate limiter (`TokenBucketRateLimiter`). Each source has a dedicated instance consumed inside `AIService` before each network call: Wikipedia at 10 req/s (burst 20), arXiv at 2 req/s (burst 5), and web/DuckDuckGo at 1 req/s (burst 3). A token bucket was chosen over a fixed-window counter because fixed windows can allow 2× the quota at window boundaries, and over a leaky-bucket queue because silently queuing excess requests increases latency unpredictably.

LLM tokens-per-minute limiter (`LlmTpmLimiter`). Provider rate-limit contracts (Anthropic, OpenAI, Google) are expressed in tokens-per-minute, not requests-per-minute. A single synthesis call that produces 800 completion tokens consumes the same quota as eight 100-token calls; request-rate limiting alone cannot prevent `429 RateLimitError` responses from the LLM provider under heavy load. `LlmTpmLimiter` uses the same sliding token-bucket mechanic, but its bucket capacity is `tpm_limit` tokens and its refill rate is `tpm_limit / 60` tokens per second (i.e. the full budget refills in exactly one minute, matching the provider contract). After each successful `AIService.synthesize` call, actual prompt + completion token counts are read from the provider's usage metadata and deducted from the rolling budget; the module falls back to `max_tokens` (1 024) when usage data is absent, ensuring the limiter is never silently bypassed. The module-level `llm_tpm_limiter` defaults to Anthropic's free-tier ceiling (40 000 TPM).

If the provider returns HTTP 429 with a `Retry-After` header, the `RateLimitError` subclass carries the server-supplied delay and the `_before_sleep` callback overrides tenacity's computed wait with the header value—respecting the provider's own back-off window exactly. During development, DDG rate-limiting was triggered on rapid repeated queries; the `ResilientDuckDuckGoProvider` fell through to its `html` backend successfully.

4. Robustness & Error Handling

4.1 Wrapping the AI module

`AIService` wraps all four `ai.*` functions with three cross-cutting concerns.

Retries. Fetch methods use a tenacity decorator from `_make_retry(max_attempts=4)` with `retry_if_exception_type(_TRANSIENT)`, `wait_exponential(multiplier=1, min=1, max=30)`, `stop_after_attempt(4)`, and `reraise=True`. The `_TRANSIENT` tuple covers `ConnectionError`, `TimeoutError`, `OSError`, `ProviderError` (and its `RateLimitError` subclass), and `RateLimitExceeded`. `ValueError` and `TypeError` are not retried—they indicate

programming errors that would fail identically on every attempt. Synthesis uses an explicit `for`-loop retry because tenacity's async retry decorator requires special executor integration.

Timeouts. Every fetch is wrapped in `asyncio.wait_for(coro, timeout=settings.X_timeout)`, converting `asyncio.TimeoutError` to a `TimeoutError` already in `_TRANSIENT`. Synthesis has a hard 60-second wall-clock deadline; a `TimeoutError` from synthesis propagates to the caller without retry.

Logging. Every method emits structured `logger.info/warning/error` calls using `extra={...}` dicts. Fields at the `INFO` level include `query`, `count`, and `elapsed_s`; retries log at `WARNING` with `attempt` and `retry_after_s`; hard failures log at `ERROR`. No `print()` calls exist in `src/`.

Input validation. Before any network call, `ResearchRequest` (Pydantic) enforces non-empty stripped question, max 500 characters, and sources restricted to {`wiki, arxiv, web`}. On the response, the `ai/` module drops out-of-range citation indices; our layer additionally strips ANSI escape sequences and caps the answer at 4000 characters via `_sanitize_text`.

Listing 2: Retry factory with server `Retry-After` awareness

```python
def _make_retry(max_attempts: int = 4):
    def _before_sleep(retry_state):
        exc = retry_state.outcome.exception()
        if isinstance(exc, RateLimitError) and exc.retry_after is not None:
            retry_state.next_action.sleep = exc.retry_after  # honour hint
        else:
            before_sleep_log(logger, logging.WARNING)(retry_state)

    return retry(
        retry=retry_if_exception_type(_TRANSIENT),
        wait=wait_exponential(multiplier=1, min=1, max=30),
        stop=stop_after_attempt(max_attempts),
        before_sleep=_before_sleep,
        reraise=True,
    )
```

4.2 Failure-mode analysis (one concrete incident)

During early integration testing the arXiv source failed on every request. The failure mode was: `AIService.fetch_arxiv` raised `ConnectionError` on every attempt, tenacity exhausted its four retries (backoff $1\,\text{s} \to 2\,\text{s} \to 4\,\text{s}$), and the exception was captured by the orchestrator's `return_exceptions=True`. The `ResearchResult` returned with `sources_failed=["arxiv"]` and a complete answer synthesised from Wikipedia and web sources. The CLI displayed a `[!] Sources that failed: arxiv` line below the answer.

Inspecting the `WARNING` log revealed `source_fetch_failed` entries with `{source: arxiv, error: "Connection refused"}` appearing four times per question. The root cause was an `http://` endpoint in the upstream provider configuration instead of `https://`—the provider raised `ConnectionError` because the plaintext HTTP redirect was blocked. Correcting the scheme to `https://` restored normal operation (`sources_failed` became empty). This incident confirmed that the degradation path works end-to-end and that the structured log was sufficient to diagnose the issue without rerunning the code.

4.3 Input validation

Every entry point validates before touching the network:

- **CLI (`ask` command)** — `ResearchRequest` rejects blank questions (after strip), questions exceeding 500 characters, and invalid source names. The `_parse_sources` function raises `click.BadParameter` for unknown source tokens; `ask` catches `ValidationError` and exits with code 1, printing the message to stderr.

- **Source filter** — `SourceFilter` is a `str(Enum)`; any token not in {`wiki`, `arxiv`, `web`} raises immediately.
- **Output sanitisation** — `_sanitize_text` strips ANSI escapes (preventing terminal injection) and caps output at 4000 characters (preventing runaway LLM responses).
- **Environment** — `pydantic-settings` validates all typed fields at startup: `max_concurrent` is bounded [1, 50]; timeouts are `float > 0`. Missing required API keys cause a `ProviderError` at first use, not at import time.
- **Database** — the repository's `db_url_is_placeholder` check detects an unconfigured `DATABASE_URL` and skips pool creation, preventing a startup failure when only the research functionality (not history) is needed.

5. Storage & Caching

5.1 Schema

The PostgreSQL schema for session persistence is defined in `src/storage/repository.py`:

Listing 3: PostgreSQL schema for `research_sessions`

```
CREATE TABLE IF NOT EXISTS research_sessions (
    id              SERIAL PRIMARY KEY,
    question        TEXT    NOT NULL,
    answer          TEXT    NOT NULL,
    citations_json  JSONB   NOT NULL DEFAULT '[]',
    sources_failed  JSONB   NOT NULL DEFAULT '[]',
    cached          BOOLEAN NOT NULL DEFAULT FALSE,
    elapsed_seconds FLOAT   NOT NULL DEFAULT 0.0,
    created_at      TIMESTAMPTZ      DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_research_sessions_created_at
    ON research_sessions (created_at DESC);
```

`citations_json` and `sources_failed` are stored as `JSONB` rather than plain `TEXT` for two reasons: PostgreSQL can index and query JSONB server-side, and JSONB validates structure on insert so a malformed citation list is rejected at the database level. `created_at` uses `TIMESTAMPTZ` (timezone-aware) to avoid ambiguity in multi-region deployments. The descending index on `created_at` makes the `history` query (order by recency, limit 20) a fast index scan.

5.2 Caching policy

The TTL cache (`src/services/cache.py`) stores `list[Source]` objects keyed by `(source_name, canonical_query)`, where the canonical query is lowercased and whitespace-normalised. Entries are stored in a module-level `dict` and evicted lazily on read if their age exceeds `CACHE_TTL_SECONDS` (default 3600 s; 0 = never expire). The `-no-cache` flag bypasses both reads and writes for the current request.

Cache hits and misses are per-source: if Wikipedia is cached but arXiv is not, only arXiv is fetched, and the results are merged before synthesis. This fine-grained partitioning maximises cache utility when the user reruns with a different `-sources` subset.

From the offline demo run (`artefacts/demo_offline_run.txt`), repeated identical questions yield a 100% cache hit rate on the second pass. In production the hit rate depends on query diversity and TTL; a 1-hour TTL suits most research topics, but time-sensitive queries should use `-no-cache`.

Stale-prone entries include web-search results for news queries and arXiv results when new papers are published on the same topic; there is no active eviction beyond TTL.

6. Testing

6.1 Strategy & coverage

All 127 tests run without any real network access. The `conftest.py autouse` fixture patches `ai.fetch_wikipedia` and `ai.fetch_arxiv` to deterministic async stubs returning canned `Source` objects. Web search is injected via `FakeWebSearch`, a `WebSearchProvider` subclass that records calls without hitting the network. LLM synthesis is replaced by `FakeLLM`, a `LLMProvider` that records every prompt and returns a deterministic response with `[1]` and `[2]` citation markers.

Module	Coverage
`src/concurrency/orchestrator.py`	100%
`src/config.py`	100%
`src/models.py`	100%
`src/logging_config.py`	100%
`src/storage/repository.py`	91%
`src/services/cache.py`	89%
`src/core/researcher.py`	88%
`src/cli.py`	90%
`src/services/ai_service.py`	77%
`src/services/web_provider.py`	77%
`src/services/rate_limiter.py`	75%
TOTAL (src/)	**87%**
Provided `ai/` smoke tests	**16/16 passing**

Coverage was measured with `pytest -cov=src -cov-report=term-missing` (Python 3.13, 127 tests, 50.34 s). The 60% minimum is met by a comfortable margin; the gap to 100% is concentrated in the retry-exhaustion paths of `ai_service.py` (all four attempts must fail in sequence), the DuckDuckGo backend-cycling paths in `web_provider.py`, and the psycopg3-unavailable branches in `repository.py` (which require a live Postgres process).

6.2 Notable tests

Happy-path end-to-end (`test_concurrency.py::test_fetch_sources_all_succeed`). Calls `fetch_sources_concurrent` with all three sources and the `FakeWebSearch` provider, asserting that all three source names appear in `fetched` and `failed` is empty. This exercises the full orchestrator path—semaphore acquisition, coroutine dispatch, result collection—with zero real I/O.

Concurrency test (`test_concurrency.py::test_semaphore_limits_concurrency`). Spawns 10 tasks through `run_pipeline` with `max_concurrent=3` and tracks the running count with a shared integer. Asserts `peak_running` $\leq$ `3`. This test would catch a regression where the semaphore was accidentally removed—a real bug that is invisible to coverage-only analysis.

Error-path test (`test_concurrency.py::test_fetch_sources_graceful_degradation`). Monkeypatches `_FETCH_FNS["arxiv"]` to an async coroutine that raises `ConnectionError`, then asserts `"arxiv" in failed` and `"wikipedia" in fetched`. This test surfaced a real design issue: an early version of the orchestrator used `asyncio.gather` without `return_exceptions=True`, causing a single source failure to silently drop the Wikipedia result. Adding `return_exceptions=True` fixed the bug and this test guards against regression.

The test we most wished we had time to write is a true integration test that spins up a local HTTP mock server (via `respx`) and validates the full pipeline from CLI invocation to rendered output,

including the retry-backoff timing—current CLI tests mock at the `run_research` level and do not exercise the retry/timeout interaction end-to-end.

7. Deployment & Reproducibility

7.1 Docker image

The project uses a **multi-stage Dockerfile**: a `builder` stage (based on `python:3.12-slim`) creates a virtual environment and installs all dependencies; the runtime stage copies only `/opt/venv`, keeping the final image free of `gcc`, pip cache, and build tools.

Listing 4: Build and run commands

```
# Build (linux/amd64 for CI compatibility)
docker build --platform linux/amd64 -t researcher .

# Offline demo (no API keys required -- default CMD)
docker run researcher

# Interactive with real providers
docker run --env-file .env researcher \
    python -m researcher ask "What␣is␣photosynthesis?"

# JSON output
docker run --env-file .env researcher \
    python -m researcher ask "quantum␣computing" --json-output

# Full stack with PostgreSQL history + Streamlit GUI
docker compose up --build
# Then open http://localhost:8501 for the GUI
```

The runtime image is built on `python:3.12-slim` (∼130 MB base) to minimise attack surface. A non-root `appuser` is created via `useradd` before files are copied and before `USER appuser` is set, so no runtime process runs as root. The CI pipeline (`.github/workflows/ci.yml`) runs a `docker build` step on every push to verify image buildability.

7.2 Configuration

All configuration is read via `src/config.py` (`pydantic-settings`); no module reads `os.environ` directly except the mirror function in `config.py` itself. The table below documents every environment variable:

Variable	Default	Effect if missing / default
LLM_PROVIDER	anthropic	Selects backend (anthropic|openai|gemini)
LLM_MODEL	claude-sonnet-4-6	Model identifier sent to provider
ANTHROPIC_API_KEY	""	Required for Anthropic; ProviderError on first use if unset
OPENAI_API_KEY	""	Required for OpenAI
GOOGLE_API_KEY	""	Required for Gemini
WEB_SEARCH_PROVIDER	duckduckgo	Backend (duckduckgo|tavily|serper)
TAVILY_API_KEY	""	Required if WEB_SEARCH_PROVIDER=tavily
SERPER_API_KEY	""	Required if WEB_SEARCH_PROVIDER=serper
DATABASE_URL	placeholder	If placeholder/unset, history is skipped gracefully
LOG_LEVEL	INFO	Root logging level
MAX_CONCURRENT	5	Semaphore bound [1, 50]
CACHE_TTL_SECONDS	3600	0 = never expire
WIKIPEDIA_TIMEOUT	10.0	Per-call timeout (seconds)
ARXIV_TIMEOUT	15.0	Per-call timeout (seconds)
WEB_TIMEOUT	10.0	Per-call timeout (seconds)
MAX_QUESTION_LENGTH	500	Pydantic rejects longer questions
MAX_SOURCES	9	Sources passed to LLM (caps token usage)

7.3 CI/CD pipeline

The GitHub Actions workflow (`.github/workflows/ci.yml`) runs on every push and pull request to `main`. Steps in order: checkout, Python 3.13 setup with pip cache, dependency install, `TODO/FIXME` guard (fails CI if any markers remain in `src/`), Ruff lint, mypy type check, pytest with coverage enforcement (≥60%), smoke tests, and Docker build.

Two pragmatic adjustments were made to the CI configuration. First, the `ruff` lint step uses `-ignore F401` (unused imports): the provided smoke-test files import symbols solely to verify that they are importable, triggering `F401` on lines that are correct by design. Silencing `F401` globally was preferable to adding `# noqa` comments to instructor-provided files. Second, the `mypy` step runs with `-ignore-missing-imports` and `continue-on-error: true`: the provided `ai/` module (which must not be modified per the project specification) contains type annotations that are incompatible with the pinned versions of the `anthropic` and `openai` SDKs, producing non-actionable `arg-type` and `union-attr` errors. These are static-analysis artefacts from the provided codebase, not type errors in our `src/` code; all runtime tests pass cleanly. The distinction between tooling noise and actual failures is visible in the CI log: the `pytest` step exits 0 while the `mypy` step may exit non-zero.

8. Limitations & Future Work

- **In-memory cache is process-local and ephemeral.** Each process restart begins with a cold cache. Running two Docker replicas gives each worker an independent cache island with no cross-process coherence. Replacing `InMemoryCacheBackend` with a Redis-backed implementation (same `CacheBackend` ABC) would fix this without touching callers.

- **LLM synthesis executor is the new bottleneck above 4 concurrent questions.** `ai.synthesize()` is dispatched to a `ThreadPoolExecutor(max_workers=4)`. Beyond four simultaneous questions the executor queue grows, partially serialising responses. Tuning

`_SYNTH_POOL_SIZE` or switching to `asyncio.to_thread` (if the provider SDK releases the GIL) would extend the linear scaling region.

- **DuckDuckGo web search quality and availability varies.** DDG is the keyless default but returns lower-quality snippets than Tavily and occasionally rate-limits aggressively. Users who need research-grade web results should configure `WEB_SEARCH_PROVIDER=tavily`.
- **Rate limiters are not shared across processes.** Each worker maintains its own token bucket. Under multi-replica Docker deployments the effective aggregate rate is `rate` × `num_replicas`, which may exceed arXiv's 3 req/s guideline. A Redis-backed distributed rate limiter (`INCR`/`EXPIRE`) would be required for multi-worker production.
- **No load testing has been performed.** The benchmark covers N=5 questions in one process. Under sustained load (more than 50 concurrent users) unknown failure modes may emerge—particularly around the PostgreSQL connection pool (`max_size=5`) and arXiv rate limits. Profiling under load before any production deployment is essential.

9. Tools & Acknowledgements

Claude (Anthropic) was used to draft the initial structure of `ai_service.py` (the retry decorator factory and executor-based synthesis loop), `orchestrator.py` (the `_guarded` coroutine pattern), and `conftest.py` (the `FakeLLM` and `FakeWebSearch` class skeletons). In each case the team reviewed the output line by line and made substantial revisions: the `_before_sleep` callback for `Retry-After` header handling was added after observing that the draft did not handle provider-supplied wait hints; the monkeypatch strategy in tests was redesigned to intercept the `_FETCH_FNS` dispatch dictionary rather than patching module-level functions; and the `asyncio.wait_for` vs. `asyncio.timeout` decision was changed to `wait_for` throughout for Python 3.10 compatibility after the team verified the runtime environment. Architecture decisions—why asyncio, why a semaphore, why per-source timeouts inside `_fetch_one`—are the team's own.

GitHub Copilot was used for boilerplate completion (import statements, docstring formatting, `__repr__` methods). No generated code was accepted without reading.

References

[1] Anthropic API Documentation. *claude-sonnet-4-6 model card and API reference.* https://docs.anthropic.com/

[2] tenacity library. *Retrying library for Python.* https://tenacity.readthedocs.io/

[3] pydantic-settings. *Settings management using Pydantic.* https://docs.pydantic.dev/latest/concepts/pydantic_settings/

[4] psycopg 3 documentation. *Async support and connection pools.* https://www.psycopg.org/psycopg3/docs/

[5] Python asyncio documentation. *High-level API and synchronisation primitives.* https://docs.python.org/3/library/asyncio.html

[6] Click documentation. *Composable command line interface toolkit.* https://click.palletsprojects.com/

A. Appendix — Reproducing the benchmark

Exact commands to reproduce the Section 3 benchmark. Anyone with the repo and a valid `.env` should be able to run the lines below and get numbers within 20% of the table.

```
# Clone and install
git clone https://github.com/abdurahmanligulnisa/research-assistant.git
cd research-assistant
pip install -r requirements.txt

# Configure API keys
cp .env.example .env
# (edit .env: fill in LLM_PROVIDER, API key, WEB_SEARCH_PROVIDER)

# Offline benchmark -- no API keys required, validates scheduling:
python scripts/benchmark.py --n 5 --offline

# Live benchmark -- requires valid API keys (both modes back-to-back,
# caches cleared, same process, same machine):
python scripts/benchmark.py --n 5

# Or via Docker:
docker build --platform linux/amd64 -t researcher .
docker run --env-file .env researcher python scripts/benchmark.py --n 5

# Run full test suite with coverage:
pytest --cov=src --cov-fail-under=60 --cov-report=term-missing -v

# Run only smoke tests (no API keys needed):
pytest tests/test_ai_smoke.py -v
```

Benchmark environment (`artefacts/bench_results.txt`): Python 3.13.3, pytest 8.2.2, Windows 10, live network, `LLM_PROVIDER=anthropic`, `WEB_SEARCH_PROVIDER=duckduckgo`, caches cleared before each mode, `MAX_CONCURRENT=5`.

End of report — thank you for reading.